

The stuff you save to get better at your job, and never look at again.

How I designed and built an AI tool for the things you save to learn from. The first version is for people studying product management. Nothing underneath it is tied to that.

Everyone saves things they mean to learn from. Almost nobody goes back. PM Content Vault takes the posts, PDFs, screenshots and chat exports sitting in five different apps, works out what each one is, and lets you ask questions of the whole pile. Every answer shows you which saves it came from.

Role: Product & build, end to end · **Status:** Live

LIVE APP pm-content-vault.vercel.app · **SOURCE** github.com/RADHIKA2909/pm-content-vault
Read this online at pm-content-vault.vercel.app/prd-case-study

Try it without signing up: demo@pmvault.app / [PMVaultDemo2026!](#)

A demo account with one saved item of every supported type.

The problem

Anyone learning on their own builds up a pile of saved stuff. A framework from LinkedIn. A PDF someone dropped in a group chat. A screenshot. An article bookmarked at midnight. All saved on purpose, and almost none of it opened again.

The easy answer is that people are messy and need better folders. I don't buy it. Saving isn't the broken part. Three things break after it.

It's scattered

LinkedIn, WhatsApp, bookmarks, a notes app. No one place to look, so looking anywhere feels pointless.

You can't judge it

A saved link is a title and a thumbnail. Finding out if it's worth ten minutes takes ten minutes.

You can't find it

You remember *"that post about handling stakeholder conflict"*. That's never a word that actually appears in it.

THE IDEA THIS PRODUCT IS BUILT ON

Storing things was never the problem. **Finding them and judging them** was. So a folder app solves a problem I don't have. That's why every item gets a summary written the moment it's saved, and why search came before any tidying features.

Who it's for

Long term, anyone who saves things to learn from later. People studying for exams, switching careers, or teaching themselves something new. The habit is the same everywhere: save it, never go back.

The first group is **people learning product management**. The problem isn't special to them, but they share a vocabulary, a deadline and fairly predictable content. That makes the categories worth having and the users easy to find and talk to. It's where the product starts, not where it stops.

WHY START NARROW WHEN THE PROBLEM IS WIDE

A fixed set of categories is what makes a saved pile scannable, and you can only pick good ones if you know the subject. Build for everyone on day one and you get vague categories, which pushes the sorting work back onto the user. That's the thing that already failed. **Opening it up later is a settings change, not a rebuild.**

Why build it when so much already exists

Lots of tools store saved content. None of them fix the exact thing that was broken for me.

WHAT I COULD HAVE USED	WHY IT DOESN'T FIT
A notes app	Great storage, but sorting is your job. That's the step people skip while mid-scroll. Any tool that asks me to sort brings the problem back.
A read-later app	Built for articles and a reading queue. Most of what I save is a screenshot, a PDF or a chat message. It answers "what haven't I read", not "where's that thing about stakeholder conflict".
A highlights tool	Good at pulling quotes out of things you already read. It assumes you know which source to open. My problem starts before that.
A general AI chatbot	Happily answers questions about a file you give it, but every chat starts empty. Nothing builds up, nothing gets checked for duplicates, and nothing remembers what you've read or starred.
Platform search	Saved posts stay on the app that hosted them. That's five searches instead of one, all keyword-only, none aware of the others.

THE GAP THIS FILLS

Each option above is good at one of three things: *take anything, understand it without being asked, answer across all of it with sources*. The value is doing all three to the same pile. They also depend on each other, because if nothing is understood at save time, there's nothing to search later.

The journey, and where it still hurts

Four moments matter. Here's what the product does at each, and what's still annoying.

- 1 Spot something worth keeping**

On a phone, inside LinkedIn or a group chat. You have a few seconds. Anything slower than that and the thing is gone.

Still annoying: copy → switch app → paste

The one step the MVP doesn't fix, which is why sharing straight from other apps is first on the roadmap.
- 2 Save it and stop thinking**

Paste, upload, or drop in a chat export. That's the whole job. The app does the reading and sorting, and shows you what it got before anything is saved.

No filing, no folders, no naming
- 3 Come back days later, half-remembering**

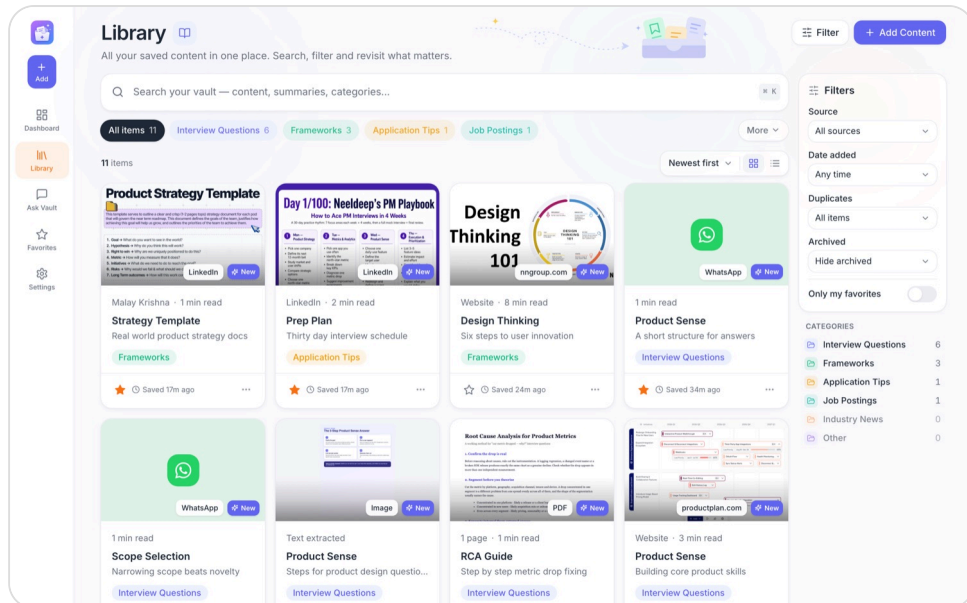
Browse the Library, where each card shows a gist, a type and a read time. Or ask a question in plain English and let search find it.

Search by meaning, not keywords
- 4 Check whether to trust the answer**

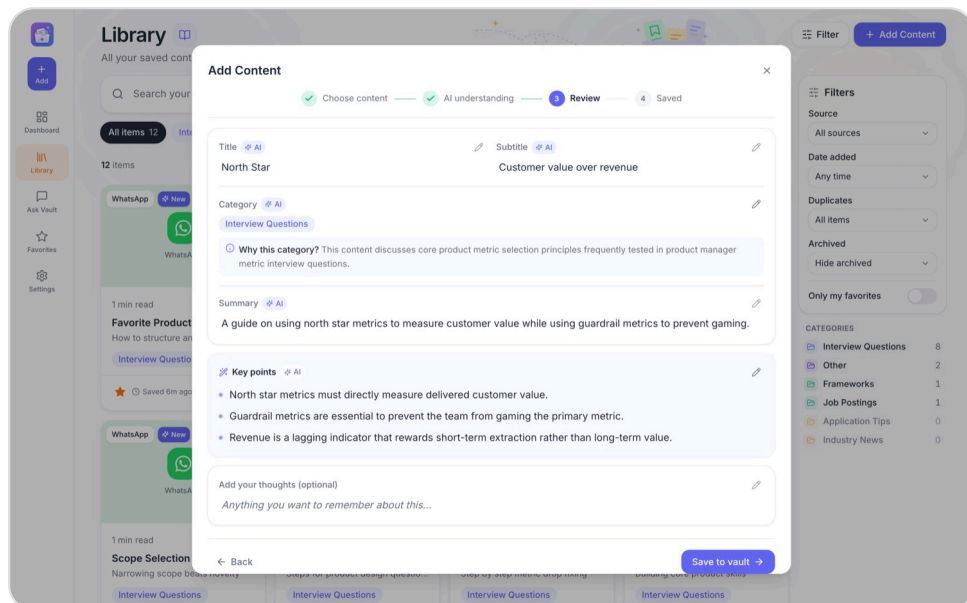
Every answer is numbered back to the saves it used, and opening one shows the exact passage behind it.

Every claim traceable to a source

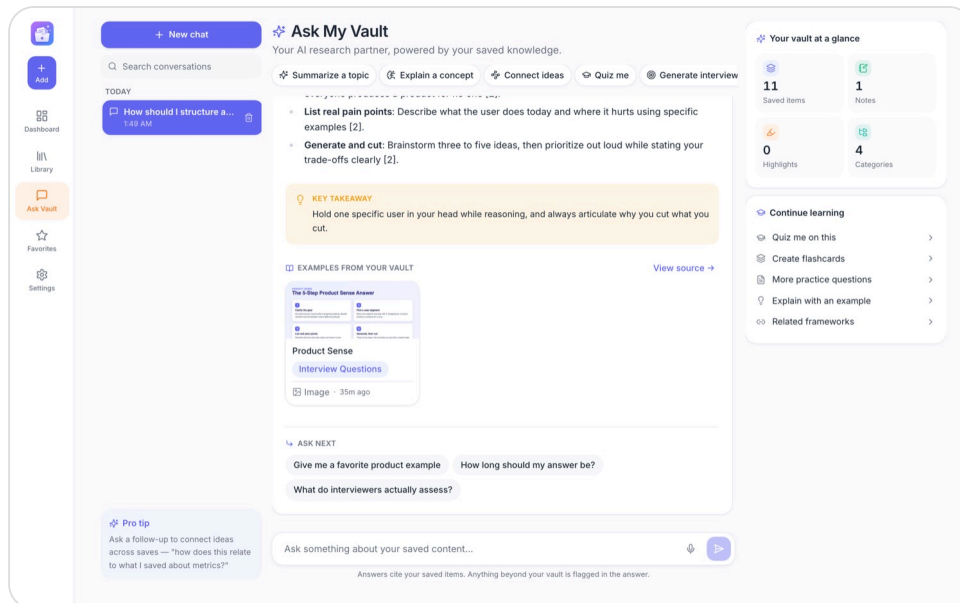
What it does



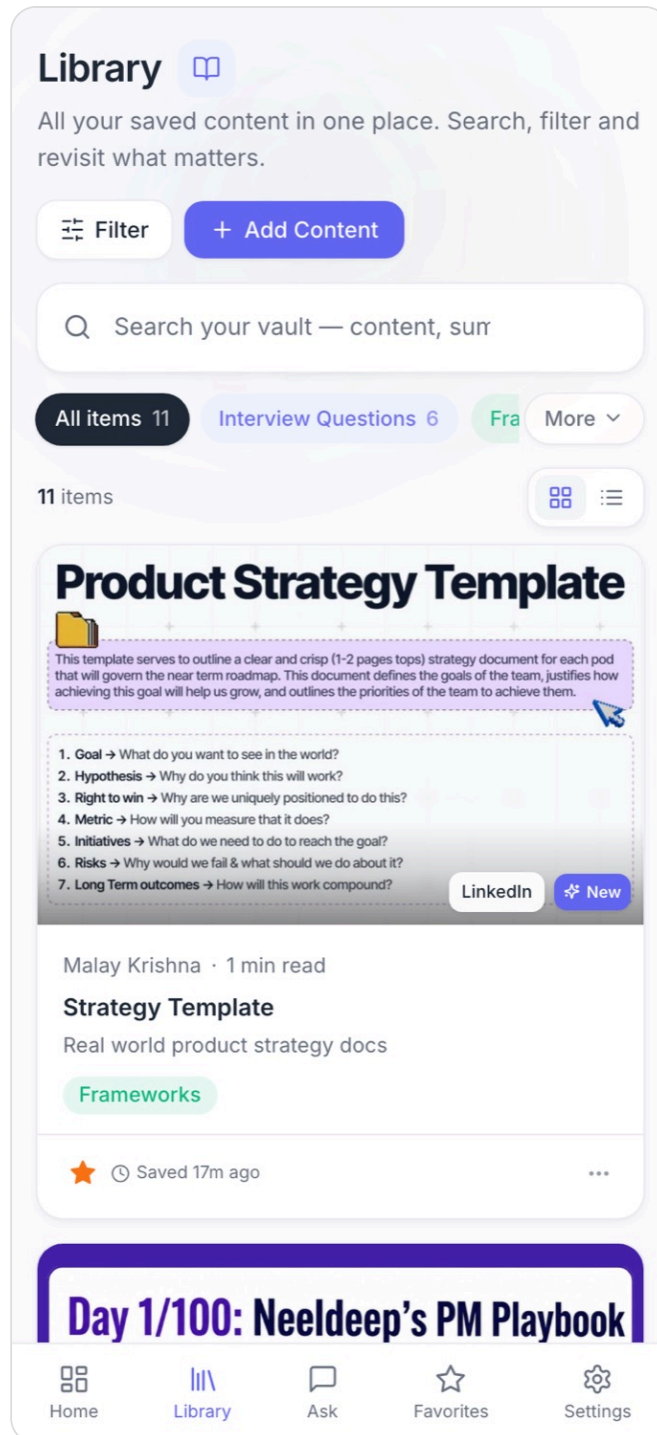
Every card knows what it is. A PDF shows its first page, a screenshot shows the image, a LinkedIn post shows the author. No generic file icons, because you should recognise a save without reading it. If a card has no page count or read time, it leaves the line out instead of showing a zero.



The AI suggests, you approve. Adding something reads it, sorts it, summarises it and checks for duplicates, then stops and shows you the result. Every field is editable. An **AI** chip marks what the model wrote and disappears once you change it.



Answers you can check. Ask My Vault answers from your own saves, with numbered sources. Open a source and you see the exact passage the answer used, not the item's summary.



Built for where saving actually happens. The sidebar becomes a bottom bar and cards stack into one column.

How it's organised

Five places to go, plus one decision underneath that everything follows from.

Dashboard	: counts, recent saves, continue a conversation
Library	: everything, filtered by category / source / date / duplicates
↳ Item	: the content, its summary, highlights, related items
Ask My Vault	: conversations, each answer linked back to the Library
Favorites	: starred items, with study actions over them
Settings	: account, defaults, data

One message is one item

A WhatsApp export doesn't save as one item. Each message becomes its own. Otherwise a 40-message chat with three good ideas in it is one big unsearchable blob, and a search hands back the whole conversation when you wanted one paragraph. Splitting it up front is what lets a source link point at something small enough to check.

Two kinds of label, on purpose

	CATEGORIES	TAGS
Who sets them	The AI picks from a fixed list. You can override it	You, freely
How many	One to three per item, never zero	Any
Why	A fixed list keeps filtering reliable. An open one gives you "Product Sense", "Product-Sense" and "PS" inside a week	Your own structure, for things the fixed list can't cover. Favourites are built as a tag

NEVER ZERO

Every item gets at least one category, falling back to `Other` if the model won't pick. An "uncategorised" pile sounds harmless, but it's exactly where things go to be forgotten, and forgetting is the problem this whole product exists to fix.

Decisions that shaped it

The feature list is the least interesting part. What got argued down is where the thinking shows.

DECISION 01

Search by meaning, not keywords, and no model training

Keyword search fails on the query people actually have, because you remember a post's *idea*, not its words. Training a model on your own library has a different problem: your library changes daily, so every new save would mean retraining. Search by meaning solves both. A new save is findable the second it's added, and every answer can point back at what it came from. That became the tiebreaker for later calls: anything trading source accuracy for convenience got dropped.

DECISION 02

Nothing is saved until you confirm

The normal way to build this is to save the row first and tidy it up in the background. I did the opposite. All the AI work runs in memory, and only pressing save writes anything. Two reasons. An abandoned add should leave nothing behind, since not accumulating junk is half the point. And a duplicate warning is only useful *before* you save. Telling someone they've just made a duplicate is a notification, not a feature.

DECISION 03

No LinkedIn scraping, even though it's the biggest source

Most of what I save is on LinkedIn, and auto-importing it would be the single most useful feature. It also breaks LinkedIn's terms, so the app only takes a pasted link or pasted text. This costs real convenience. I wrote it into the brief as a locked rule before building, so I couldn't quietly talk myself out of it at 1am when the manual flow felt slow.

DECISION 04

Cutting a feature I'd already planned

"Resurfacing" was feature #7 in my own brief: anything untouched for 14 days pops up in a reminder widget. Six of seven shipped. I cut this one, and not because it was hard. It was the cheapest thing left, two rules over a date. I cut it because **showing you something just because it's old is a guess about what you want, and the product has no evidence for that guess.** "You

saved this two weeks ago" isn't a reason to read it. It goes back on the list the day users show me they're hunting for forgotten saves, and I've written down what that evidence looks like.

DECISION 05

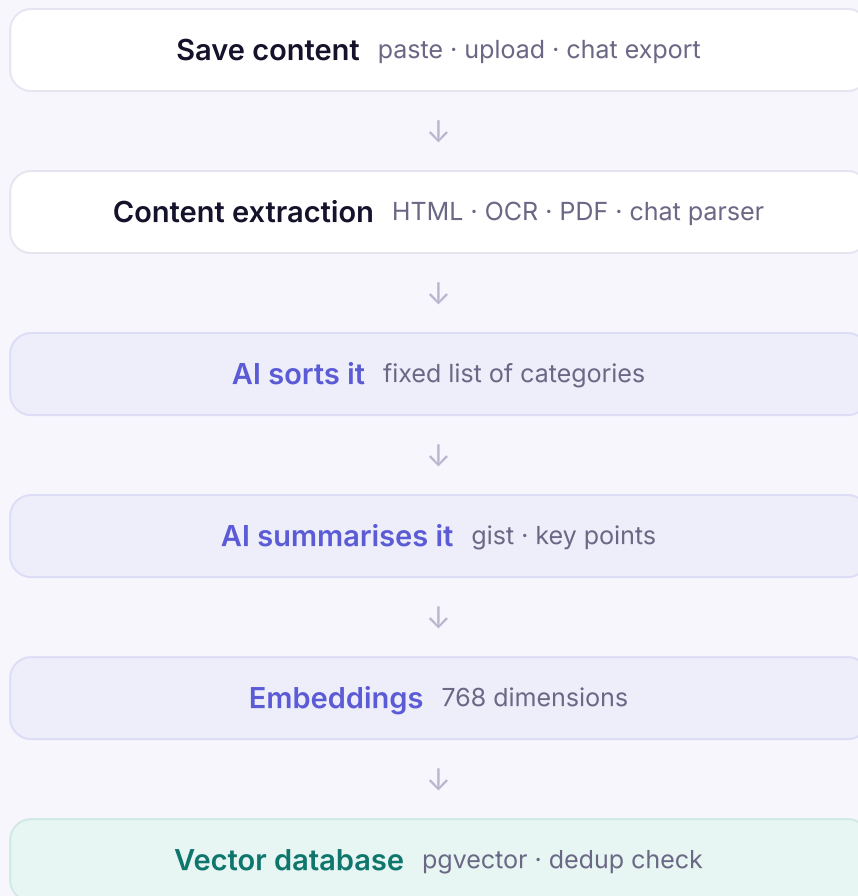
A hard \$0 budget, treated as a design constraint

Everything runs on a free tier. Supabase for the database, auth and files. Vercel for hosting. Gemini for the AI. Reading text out of images and PDFs happens in the browser, where it costs nothing. That shaped the build, not just the bill. Uploads cap at 4MB because the hosting rejects anything bigger before my code runs, so a higher limit would only swap a clear message for a confusing failure. A daily ping keeps the database awake, because free projects go to sleep and a portfolio link is most likely to be opened by the exact person you don't want waiting.

AI architecture

Two AI paths, kept apart on purpose. One runs when content arrives, the other when a question is asked. They share an embedding model and nothing else.

INGESTION: WHEN CONTENT ARRIVES



RETRIEVAL: WHEN A QUESTION IS ASKED



What each stage does

- **Reading the content depends on the type.** Notes come in as clean HTML. Screenshots go through Tesseract in the browser. PDFs go through `pdf-parse`, with page 1 rendered as the cover image. Chat exports get split into one item per message. Links get fetched.
- **Sorting and summarising happen in one call.** Gemini returns a category from the fixed list, a title, a short summary and 3 to 5 key points. The reply is checked, not trusted. Broken JSON gets repaired, and an empty reply throws an error instead of quietly saving a blank item.
- **The duplicate check reuses the same maths**, and runs *before* saving, while it can still change your mind.

- **Only sources the answer actually used get listed.** Search returns the top matches, and the answer uses some of them. Listing all of them would make "used 5 sources" a lie.

Designing for the model failing

The model is treated as unreliable, because it is.

WHAT GOES WRONG	WHAT HAPPENS
Model overloaded	Retry, then try other models. The cheap one first, the expensive one last, since it only allows about 20 requests a day free
Out of daily quota	A different error message, because the advice differs: wait a minute versus wait until tomorrow
Broken reply format	Repaired field by field instead of thrown away
Sorting fails completely	The save still works. You get the item with blank fields you can fill in. The AI speeds this up, it doesn't block it
Search indexing fails	Retried. An item with no index is invisible to search and skips the duplicate check, which is worse than a missing summary you can see

THE RULE UNDERNEATH

All of these fall back to the same thing: *you still get your content saved*. The one thing that must never slip is the source link. An answer you can't check is worse than no answer, because it looks exactly like one you can.

What broke, and what I changed

Four failures worth writing down. Each changed how I think about shipping AI features.

– The app blamed the user for Google's outage

Saving started failing with *"The AI couldn't read this one."* The content was fine. Google's model was returning "busy, try later" on five calls in a row.

Three mistakes stacked up. One model with no backup. No retry. And no logging, so an outage at Google's end looked exactly like unreadable content, with nothing to tell them apart. Worst of all, the message blamed the user for someone else's traffic spike.

Now: it retries, falls back to other models, logs every failure, and says "the service is busy" separately from "you're out of quota", because the advice is different.

– A source link pointing at the wrong number

Sources are numbered by where they came up in the search, so an answer using only the fourth result correctly says [4]. But reopening an old conversation renumbered them from 1, so the text said [4] next to a card labelled 1.

Worse, it showed the item's summary instead of the passage the answer actually used. For a product whose whole promise is checkable answers, that's the failure that matters most.

Now: the number and the exact passage are saved with the answer. The item itself is still looked up fresh, so edits show up and deleted items drop out.

– A bug only the live site would have found

Between reading your content and saving it, the text sits in memory. If that memory is missing, the app reads it again, which is sensible. But it kept only the text and dropped everything else, so a note could save **with no body at all**.

On my laptop this only happened if you left the screen open for fifteen minutes. On the live site, the two steps can run on different machines, so the rare case becomes the normal one. I caught it reading the code before deploying, so it never actually ran.

– Every uploaded file was public

Anyone with the link could open any PDF or screenshot, no login needed. Harmless on my laptop. Not harmless the moment the app had a public address.

Now: files are private and every link expires after two hours. The code reads both the old and new link formats, so nothing had to be migrated.

What I'd take to the next build

Six things I believe more strongly now. The first two and the last two were paid for by the failures above.

1. The error message is part of the feature

"The AI couldn't read this one" sent me off editing content that was fine. An error has to name the thing that actually failed, and if the system can't tell which thing that was, that's a logging problem and not a wording problem.

2. A source link is a promise

My sources looked right and were quietly wrong twice. Neither version broke a test or looked broken on screen. For a product built on checkable answers, the bar isn't "**looks cited**". It's "**can be checked**".

3. Constraints beat prioritisation frameworks

A \$0 budget and LinkedIn's terms each killed whole branches of work before I could argue with myself about them. **The rules I wrote down survived. The preferences I only held in my head didn't.**

4. Cutting a planned feature is itself a decision

The hard part of cutting resurfacing wasn't working out that it should go. It was that I'd already written it down, and written plans get heavy. Being able to explain *why* it's cut, and what would bring it back, is worth more than shipping it.

5. Working on your laptop proves less than it feels like

That empty-note bug needed a fifteen-minute pause to appear on my machine. On the live site, where each request can land on a different machine, it becomes the normal case. **Deploying isn't just a new address, it's a different environment.**

6. Lock things down before the URL goes public, not after

The public files took under an hour to fix, and it was the thing I'd have been most embarrassed to be asked about. **The moment something is reachable, its weakest assumption becomes real.**

THE ONE THAT MATTERS MOST

Most of these are the same lesson in different clothes. **The hard part of an AI product sits around the model, not inside it.** The prompt took a day. Retries, fallbacks, honest error messages, checkable sources, and deciding what to do when the model returns nonsense took the rest of the week. That's the part that decides whether anyone trusts the output.

What I'd measure, and what I'd ask

WRITTEN BEFORE THERE WAS ANYTHING TO MEASURE

Set before launch, so success can't be defined afterwards to match whatever the numbers say. Each one names **the failure it would catch**, not just the thing it counts.

The four numbers I care about

QUESTION	MEASURE	WHY THIS ONE
Does saved content get used?	% of saves opened at least once in 30 days	The main claim. If this doesn't beat roughly zero, nothing else matters.
Does search actually work?	% of questions where someone opens a source	An answer nobody clicks into is just a nice-sounding answer. That's the failure I fear most.
Is it faster than before?	Time from question to finding the right save	It only earns its place if it beats scrolling LinkedIn saves.
Is the AI trusted?	% of AI-filled fields edited before saving	Lots of edits means it's getting things wrong. Zero edits probably means nobody's reading them.

The five questions I'd ask users

1. Show me the last five things you saved. Where are they? *(Is "scattered" real, or just my problem?)*
2. When did you last go looking for something you'd saved? What did you do?
3. What happened when you couldn't find it? *(Googling it again is the strongest sign the saving was wasted.)*
4. Here's an answer using three of your saves. Do you trust it? What would make you check?
5. You haven't opened this in three weeks. Should the app have told you? *(This decides whether resurfacing comes back.)*

WHAT WOULD CHANGE MY MIND

If three of five describe hunting for something they knew they'd saved and giving up, **resurfacing goes back in**, triggered by what they were looking for and not by how old the item is.

If nobody opens a source, the trust model is wrong and answers need rebuilding around the source rather than the text.

How it's built

Frontend	React (Vite) + Tailwind, hosted on Vercel
API	Express, running on Vercel on the same domain as the site
Database	Supabase Postgres, with <code>pgvector</code> for search by meaning

Login	Supabase Auth, Google and email/password. Every table is locked to its owner
AI	Gemini for sorting, summaries and answers, plus <code>gemini-embedding-001</code> for search
In the browser	Tesseract.js reads text from screenshots, pdf.js renders PDF covers. Both free, both off the server

Two choices worth calling out. The API sits on the same domain as the site, so there are no cross-domain requests in production at all. And reading text from images and PDFs happens in the browser, which keeps it off the server and off the bill.

Product Requirements: MVP

This is the brief the build actually ran against, written before the first line of code. It's why the decisions above have dates and reasons attached.

1. Problem

People save learning material across LinkedIn, group chats and bookmarks, then rarely go back to it. Even when they do, it's hard to tell what's worth reading without reading it, or to find the right thing at all. The MVP solves this for people learning product management, where the content and vocabulary are predictable enough to sort reliably.

This shapes the build: summaries are written and stored at save time instead of on demand, and "find the right thing fast" matters more than "store everything", because a folder app already does the second one.

2. Who it's for

Long term: anyone who saves things to learn from later.

MVP: people learning product management. A big, clear first group whose content is predictable enough to make a fixed category list work.

Not yet: other roles and subjects. That's a categories and onboarding job, not a rebuild.

3. Goals and non-goals

GOALS	NON-GOALS
One place that takes every format this content arrives in	A general read-later app
Judge if an item is worth reading, without reading it	Replacing LinkedIn or WhatsApp
Answer questions across everything saved, with sources	Answering from general knowledge instead of your saves
Catch duplicates before they pile up	Sharing or multi-user features

4. Rules locked before building

- **No LinkedIn scraping.** Paste a link or the text only, because scraping breaks their terms.
- **No listening to WhatsApp.** You export a chat yourself and upload the `.txt`, which gets split into separate items.

- **Single user for now.** No sharing, but every table has an owner from day one so multi-user isn't a rewrite.
- **Resurfacing cut** (2026-08-06, after the other six features shipped). See Decision 04.

5. Feature scope

#	FEATURE	STATUS
1	Saving by hand: text, link, image (OCR), PDF, WhatsApp export, interview question, job posting	Shipped
2	Sorting and summarising at save time	Shipped
3	Embeddings stored in pgvector, so search works by meaning	Shipped
4	Library: list, filter, search, sort	Shipped
5	Ask My Vault, with every answer linked back to the saves it used	Shipped
6	Duplicate warnings when a new save is too close to an old one	Shipped
7	Resurfacing widget	Cut , see Decision 04

6. Data model

TABLE	WHAT IT HOLDS
items	The content, its summary, key points, category, dates, and type-specific fields like page count
embeddings	The maths that powers search by meaning, one per item
item_categories, tags	Categories and tags. Favourites are stored as a tag
duplicates	Pairs of near-identical items and how close they were
annotations	Highlights and notes, anchored so they survive edits to the text
chat_sessions, chat_queries	Conversations, answers, and the sources each answer used

7. Categories

Interview Questions (Product Sense · RCA · Metrics · Strategy · Behavioural) · Job Postings · Application Tips · Frameworks · Industry News · Other.

8. Not building yet

- Auto-syncing from LinkedIn or WhatsApp
- Multi-user accounts and sharing
- Resurfacing, in any form

9. What this project demonstrates

- ▶ **Defining the problem:** working out why saved content goes unused, and turning down the easy answer of better folders
- ▶ **Designing a whole AI product:** saving, reviewing, searching and checking, built as one flow
- ▶ **Designing for trust:** citations that open the exact passage used, and answers that fail honestly when the model does
- ▶ **Shipping it alone:** idea to live product, including login, security and a demo anyone can try
- ▶ **Scoping an MVP:** one clear first audience, four rules locked up front, one planned feature cut with its reason
- ▶ **Building the search itself:** text extraction, embeddings, vector search and answers grounded in what was found
- ▶ **Weighing cost against value:** a \$0 stack, OCR in the browser, and limits that fail with a clear message

10. Roadmap

None of this is built. Each one says why it isn't in the MVP.

Next: what still gets in the way

WHAT	WHY IT MATTERS
Share straight from any app	Every save today is copy, switch app, paste. Saving really happens at the share sheet on your phone, and cutting that one step matters more than anything else here, because the whole idea rests on saving staying easy.
A first-run walkthrough	A new account opens onto an empty vault. Nothing to search, nothing to ask. It should get one real item in and one question answered before it asks anyone for trust.
Export the vault	What's saved here can only be read here. Export makes the tool safe to commit to, and it belongs inside the reset and delete flows. Offering a download before wiping a library is what makes that undoable.

WHAT	WHY IT MATTERS
Browser extension	The desktop half of the same problem.

Then: past the first subject

WHAT	WHY IT MATTERS
Any job, any subject	Nothing about saving, embedding, searching or citing is specific to product management. Only the category list is. Let people set their own categories, or work them out from what they save, and this becomes a general study tool. It needs a walkthrough that asks what you're learning, and sorting that works without a fixed list.
Practice mode	The saves already hold questions, frameworks and worked answers. Turning those into drills and spaced repetition is what makes this a study tool instead of a library, and it's the best reason to come back.
Collections	Grouping saves into a plan ("week 3: metrics") gives the vault a shape that a flat list and a filter can't.

Later: needs other people in it

WHAT	WHY IT MATTERS
Shared vaults	Study groups already pass this material around in group chats. A shared vault is where export and multi-user meet.
Anonymous "what's trending"	The one network effect here. The same embeddings that power personal search would group across accounts to show what everyone is studying. Needs multi-user, a privacy model, and enough people for the average to mean anything.

WHY IN THIS ORDER

Next takes the friction out of something that already works. **Then** widens who it works for. **Later** needs other people signed up, so it can't be pulled forward however tempting. The one thing I wouldn't build without evidence is resurfacing. See Decision 04.

PM Content Vault: built and shipped as a solo project, 2026.

Live app (<https://pm-content-vault.vercel.app>) · Source (<https://github.com/RADHIKA2909/pm-content-vault>) · Demo login [demo@pmvault.app](#) / [PMVaultDemo2026!](#)